



Concurrency & Multithreading in Java

Performance Tuning & JVM Internals

Advanced Java Engineering



Session Structure

1. Thread fundamentals
2. Synchronization & Locks
3. Executor framework
4. Concurrent collections
5. JVM memory model
6. Garbage collection
7. Performance tuning
8. Real-world case studies



What is Concurrency?

Concurrency:

Multiple tasks progressing simultaneously.

Parallelism:

Tasks executing simultaneously on multiple cores.



Creating Threads

Method 1: Extending Thread

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Running...");  
    }  
}
```

Method 2: Runnable

```
Runnable task = () -> {  
    System.out.println("Task running");  
};  
  
new Thread(task).start();
```



Thread Lifecycle

NEW → RUNNABLE → BLOCKED → WAITING → TERMINATED

Understanding states is critical.

Race Condition Example

```
class Counter {  
    int count = 0;  
  
    void increment() {  
        count++;  
    }  
}
```

Multiple threads → inconsistent results.



Synchronization

```
synchronized void increment() {  
    count++;  
}
```

Ensures:

Mutual exclusion.



Synchronized Block

```
synchronized(this) {  
    count++;  
}
```

Locks object monitor.



Problems with Synchronized

- Blocking
- Performance overhead
- Deadlocks



Deadlock Example

Thread A locks Resource 1

Thread B locks Resource 2

Both waiting → deadlock.

Avoid by:

- Lock ordering
- Timeout locks



ReentrantLock

More flexible locking.

```
java.util.concurrent.locks.Lock lock =  
    new java.util.concurrent.locks.ReentrantLock();  
  
lock.lock();  
try {  
    count++;  
} finally {  
    lock.unlock();  
}
```



Volatile Keyword

Ensures:

Visibility across threads.

```
volatile boolean running = true;
```

Does NOT guarantee atomicity.



Atomic Variables

```
java.util.concurrent.atomic.AtomicInteger counter =  
    new java.util.concurrent.atomic.AtomicInteger();  
  
counter.incrementAndGet();
```

Lock-free thread safety.



Executor Framework

Better than manual threads.

```
java.util.concurrent.ExecutorService executor =  
    java.util.concurrent.Executors.newFixedThreadPool(4);  
  
executor.submit(() -> {  
    System.out.println("Task");  
});
```



Callable & Future

```
java.util.concurrent.Future<Integer> future =  
    executor.submit(() -> 10);  
  
Integer result = future.get();
```

Allows return values.



Thread Pools

FixedThreadPool

CachedThreadPool

SingleThreadExecutor

Avoid:

Creating too many threads.



Concurrent Collections

- ConcurrentHashMap
- CopyOnWriteArrayList
- BlockingQueue

THIRAVUGAL



BlockingQueue Example

Producer-Consumer pattern.

```
java.util.concurrent.BlockingQueue<Integer> queue =  
    new java.util.concurrent.ArrayBlockingQueue<>(10);
```



JVM Memory Model

Memory areas:

- Heap
- Stack
- Metaspace
- Method area



Heap Structure

Young Generation

Old Generation

Objects move during GC.

THIRAVUGAL



Garbage Collection

Major collectors:

- Serial
- Parallel
- CMS
- G1 (Default modern)



GC Concepts

Minor GC

Major GC

Stop-The-World pauses

THIRAVUGAL



Memory Leak Causes

- Static references
- Unclosed resources
- Listener leaks
- Improper cache usage



JVM Performance Tuning

Heap size:

```
-Xms2g  
-Xmx2g
```

GC tuning:

```
-XX:+UseG1GC
```




Profiling Tools

- JVisualVM
- JConsole
- JProfiler
- YourKit

Used for:

Memory analysis

Thread analysis

THIRAVUGAL



Thread Dump Analysis

Identify:

- Deadlocks
- Blocked threads
- Long-running tasks



Real World Case Study — High Traffic API

Problems:

- Thread starvation
- DB connection pool exhaustion
- Memory leak

Solutions:

- Proper thread pool sizing
- Connection pooling
- Cache optimization



Performance Optimization Checklist

- Use thread pools
- Avoid synchronization bottlenecks
- Use concurrent collections
- Profile memory
- Tune GC



Common Mistakes

- Creating threads manually
- Ignoring thread safety
- Using HashMap in concurrent code
- Ignoring GC behavior
- Not monitoring production



Interview Discussion Points

- Difference between synchronized & ReentrantLock
- What is volatile?
- Explain happens-before relationship
- How does ConcurrentHashMap work?
- How to debug memory leak?
- What is GC pause?



Concurrency Best Practices

- Prefer immutability
- Minimize shared state
- Use high-level concurrency APIs
- Avoid blocking operations



Summary

Concurrency builds:

- Scalable backend systems
- High-performance APIs
- Multi-threaded services

JVM tuning builds:

- Memory efficiency
- Low latency systems
- Production reliability

Master:

- Thread safety
- Executor framework
- JVM memory model